JOANNE HOAR

Teaching Philosophy Statement

joanne.hoar@gmail.com +1 403 854 8433 linkedin.com/in/joanne-hoar

Learning by Doing

I believe that programming is best learned the way a language is learned: by using it, making mistakes, and being gently corrected in context. My courses are built around the principle that a student who has struggled through a real problem and solved it will remember the solution; a student who has only read about it will not.

In every course I teach, the primary vehicle for learning is a sequence of graduated, applied projects. The first assignment builds the simplest meaningful thing. The second adds a layer of complexity that requires the student to refactor and reconsider the first. By the final submission, students have built something they can genuinely be proud of—and they have experienced the iterative, messy reality of professional software development, not a sanitized textbook version of it.

Live Coding as a Teaching Practice

I conduct a minimum of three live synchronous sessions per week in my online courses. The most valuable part of these sessions is not the prepared material—it is the moments when my code does not work.

When I make an error in front of students and then diagnose and fix it in real time, I am teaching something that no assignment can test: how to think when things go wrong. I narrate my reasoning aloud. I notice when I have the wrong mental model. I demonstrate that debugging is not a sign of failure—it is the work itself. Students consistently cite these moments in their feedback as the most useful parts of the course.

I maintain a public GitHub repository of course materials, sample code, and corrected examples from live sessions, so that students who learn better asynchronously can revisit any moment and study at their own pace.

Meeting Students Where They Are

A classroom is never a uniform audience. When I designed and delivered a voluntary OpenGL graphics elective at an engineering firm, my students ranged from recent graduates encountering 3D mathematics for the first time to senior developers who had shipped commercial products. I prepared materials at multiple levels: conceptual visualizations for those new to the rendering pipeline, and mathematically rigorous derivations for those who wanted to go deeper. The capstone project—building an interactive graphics widget from scratch—was the same for everyone, but the path each student took to get there was their own.

I carry this approach into every course. I ask early in each offering what students already know, what they are afraid of, and what they hope to build after graduation. These answers shape how I pace the material, which analogies I reach for, and how I frame the relevance of topics that might otherwise feel abstract.

Interaction Design as a Teaching Lens

My background in human-computer interaction is not a separate credential from my teaching—it is baked into how I design instruction itself. Every course outline is a user interface. Every assignment prompt is a communication artifact. I ask myself the same questions I would ask when designing software: What is the user's (student's) goal? What information do they need, and when? Where are the likely points of confusion, and how can I reduce them through better structure rather than longer explanations?

This lens also informs the content I bring to HCI specifically. I have spent my career building things that people interact with: games that must be immediately learnable and satisfying, developer tools that must be clear and self-documenting, augmented reality interfaces where a poorly placed affordance breaks the illusion entirely. I bring real examples from this work into the classroom—not as name-dropping, but because concrete cases are how abstract principles become memorable.

Accessibility and Inclusion

Accessible design is not an add-on—it is a signal that a designer took their users seriously. I teach this principle explicitly in interface-related courses, and I model it in my own materials: captions on recorded sessions, readable document formatting, and assignments that do not assume any particular hardware configuration. I am committed to teaching in a way that is welcoming to students with varied backgrounds, learning styles, and circumstances, and to building the habit of inclusive design into how students approach their own work from the start of their careers.

What I Am Working Toward

My goal as a teacher is not to produce students who can complete assignments—it is to produce professionals who understand what they are doing and why, and who have the confidence to figure out what they do not yet know. A student who leaves my course with the habit of reading error messages carefully, testing their assumptions, and breaking a complex problem into manageable pieces will continue learning long after the course ends. That is the outcome I am building toward, every session.